Deltazium — CDC 파이프라인 구축·운영 기록

Debezium + Kafka 기반 변경 데이터 캡처, changelog 보관·복구(replay) 아키텍처, 웹 제어 콘솔

최남희 · 단독 설계·개발 · 2026.07 – 09 · 상용 CDC 솔루션 개발 경험 기반의 검증 프로젝트

1. 개요

소스 DB의 변경(CDC)을 Debezium으로 캡처해 같은 스트림을 두 갈래로 병행 적재한다(소스·타깃은 Oracle·PostgreSQL, 캡처는 LogMiner·pgoutput). 타깃 DB에는 현재 상태를(실적재), Iceberg/S3에는 모든 변경 이력을 (append-only changelog) 쌓는다. changelog는 미러링이 아니라 특정 시점부터의 복구(replay)를 위한 원본이며, 이 복구 아키텍처가 실제 배선에서 성립하는지 증명하는 것이 프로젝트의 목적이었다.

그 위에 테이블 등록·사전 점검·DDL 승인·복구를 담당하는 제어면(Spring Boot + React)을 얹었고, 운영 중 실제 발생한 장애들을 계기로 장애 감지 → 원인 표시 → 단계형 복구, 그리고 로그·상태를 스스로 조회해 원인을 짚는 AI 진단 어시스턴트까지 운영 사이클을 확장했다.

2026.07 – 09
작업 기간 · 최남희 단독 설계·개발

소스 3
Oracle 1 · PostgreSQL 2

타깃 3
Oracle 2 · PostgreSQL 1

등록 8 테이블
changelog 테이블 8(1:1)

커넥터 14/14
running · source·jdbc-sink·iceberg-sink

자동화 테스트 136건
envelope 왕복·복구 스모크 포함

정합 검증 466,025행
컬럼별 체크섬 전량 일치(재검증)

실장애 기록 6건
원인 확정 후 대응 — docs/incidents/

규모 — Java 9.9k · TypeScript 5.3k 라인, 실트래픽 CDC 이벤트 53만+, 실장애 4건 전건 원인 확정 후 복구(유실 0).

2. 아키텍처

Kafka는 짧은 버퍼로 fan-out만 담당하고(retention 24h, 이력 보관 책임 없음), 두 sink가 독립 consumer group으로 같은 토픽을 소비해 lag를 서로 격리한다. Iceberg/MinIO가 장기 이력·복구 원본(1일 파티션 drop이 곧 보존 정책)을, Oracle 타깃이 현재 상태(PK upsert 멱등 apply)를 맡는다. 데이터 경로에는 기성 커넥터만 쓰고 자체 코드는 제어면·복구·UI에만 둔다 — apply 시맨틱을 JDBC sink 단일 경로로 유지해, 복구 결과가 라이브 적재와 달라지는 사고를 구조적으로 차단하기 위함이다.

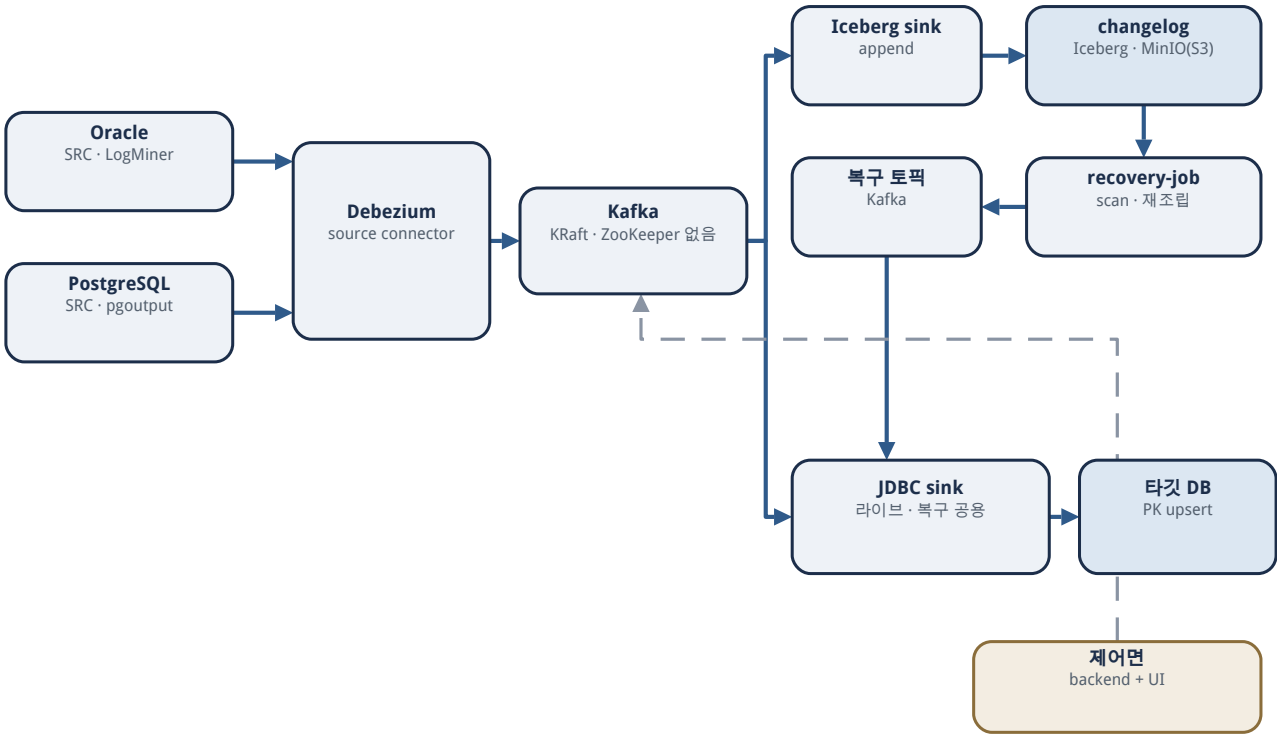


그림 A. 전체 아키텍처 — 캡처(소스별) → Kafka → 실적재(JDBC sink)·changelog(Iceberg sink) 병행, 복구는 changelog → recovery-job → 복구 토픽 → 동일 JDBC sink로 합류. 점선은 제어면의 배포·관리 경로.

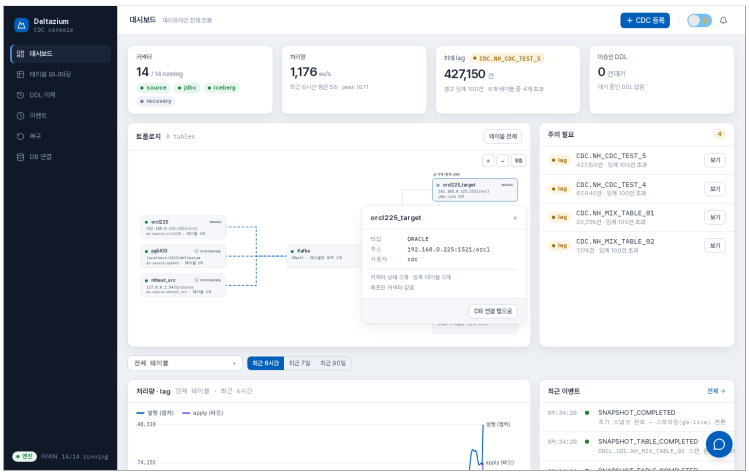


그림 B. 대시보드 동적 토폴로지(실제 화면) — 연결된 소스·타깃 수만큼 노드가 생성된다(소스 3·타깃 3). 위 SVG의 캡처·적재 구간을 실제로 배선한 화면.

설계 vs 실제. 위 SVG가 데이터 경로의 설계도라면, 왼쪽은 그 설계가 실제 배선된 모습이다. 토폴로지는 정적 그림이 아니라 등록된 소스·타깃 연결 수에 맞춰 자체 SVG로 노드를 그린다(팬·줌·노드 클릭 정보 카드, 교차 없는 직교 라우팅). 노드를 클릭하면 접속 정보·배포된 커넥터 수 같은 실측 상태가 카드로 뜬다.

3. 설계 판단

3.1 멱등 apply를 전제로 한 복구 단순화

전달 보장은 at-least-once로 두고, apply를 PK 기반 upsert(MERGE)로 통일해 몇 번을 재적용해도 결과가 같게 했다. 복구 재생의 경계 중복을 정밀 절단 없이 흡수하는 근거이며, PK 없는 테이블은 등록 단계에서 거부한다. 복구는 재발행 단일 경로 — recovery-job은 Iceberg를 읽어 복구 토픽에 재발행까지만 하고, apply는 라이브와 동일한 JDBC sink 설정이 담당한다.



그림 C. 복구 = 재발행. recovery-job은 재조립·발행까지만 하고, 타깃 apply는 라이브와 같은 JDBC sink가 맡는다 — recovery-job이 타깃에 직접 쓰지 않는다.

3.2 changelog 스키마를 실측 실험으로 확정

원안(평탄화 스키마)이 기성 커넥터·SMT 조합으로 구현 불가함을 실험으로 증명하고 (스톡 SMT는 중첩 필드 승격 불가, 기존 변환 SMT는 before 이미지 소실 — 무손실 재조립 불변식 위반), envelope 구조 그대로 저장 + backend가 테이블 사전 생성 + 1일 파티션으로 설계를 개정했다. envelope 왕복 테스트(원본 → Iceberg 행 → 재조립 동등성)가 이 불변식의 회귀 방어선이다.

필드	내용
`op`	c·u·d·r
`before`	변경 전 이미지
`after`	변경 후 이미지
`source`	소스 메타(scn·lsn) — 하류 미참조
`ts_ms`	처리 시각
`_pos`	순서·증분 기준(중립)

표 1. changelog 컬럼 — envelope-as-is + 위치 컬럼

3.3 캡처 장애의 재현·진단·전환 — 유실 0

공유 dev Oracle에서 LogMiner `redo_log_catalog` 전략이 ORA-1371(complete dictionary not found) 재시도 루프에 빠져 커넥터는 RUNNING인데 캡처가 멈추는 장애를 재현했다. 딕셔너리 아카이브 존재를 확인해 전략 자체의 취약점으로 진단하고 `online_catalog`로 전환(DDL 이벤트는 양 전략 모두 발행됨을 확인해 승인 워크플로 유지), 소스 redo 재마이닝으로 밀린 9,673건을 전량 회수했다. 커넥터 삭제 후 재등록 시 잔존 offset으로 스냅샷이 건너뛰어지는 함정도 실측으로 발견해 등록 해제 시 offset 정리를 자동화했다.

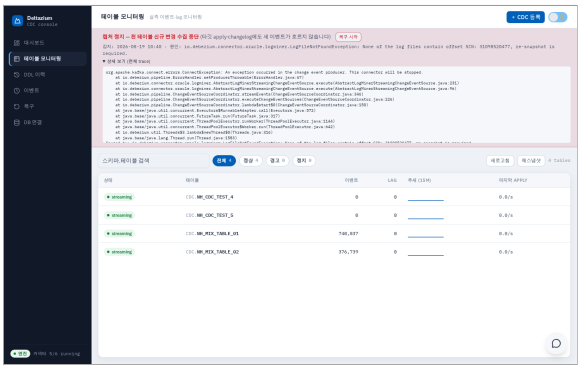


그림 D. 캡처 장애 배너(실측) — RUNNING/FAILED 불일치·원인·trace.

3.4 캡처 계정 최소 권한 8개 도출

Debezium 문서의 보수적 권한 목록(20여 개)을 실 배선의 권한 실패를 재현하며 8개로 압축했다. `SELECT ANY DICTIONARY`가 V$ 뷰 개별 grant를 대체함을 확인하고, `DBMS_LOGMNR_D`(딕셔너리 전략 전용)·`FLASHBACK ANY TABLE`(초기 스냅샷 전용)의 용도를 구분해 문서화했다. 사전 점검이 권한 누락 시 DBA용 GRANT 스크립트를 생성해 안내하고 배포를 차단한다.

3.5 운영 동작의 원칙화

jdbc-sink를 테이블별 커넥터로 분리해 타깃 이름 매핑과 테이블 단위 정지(DDL 거부 격리)를 가능하게 했고, recovery-sink는 apply 완료(consumer lag 소진)를 감지해 자동 정지한다(평시 정지 원칙). 복구 트리거의 자동 재개 옵션으로 "정지 → S3 복구 → 라이브 복귀(go-live)"가 한 번의 조작으로 완결된다 — 경계 중복은 전부 멱등이 흡수하므로 절단 시점 계산이 필요 없다.

3.6 실장애: connector RUNNING / task FAILED — 나홀 미감지의 해부와 복구

공유 dev Oracle의 archive log 정리로 재개 지점 SCN이 소실돼 캡처 task가 죽었는데, connector는 RUNNING·발행이 없으니 lag도 0 — 모든 지표가 "평온"을 가리키는 나홀이 흘렀다. Connect REST의 task trace로 원인을 확정하고 이 장애에서 기능을 파생시켰다: task 포함 상태 판정(최악값 집계), 장애 배너 (감지 시각·원인·전체 trace), 행별 재시도, 단계형 재스냅샷 워크플로. 최종 복구는 타깃 truncate 완전 재구축으로 10.5만 행을 50초에 go-live — 갭 중 소스에서 삭제된 행(고아 +2,237)까지 정리했다. "lag 0은 따라잡음과 무소식을 구분하지 못한다"는 교훈은 대시보드의 발행 vs apply 겹침 차트로 만들었다.

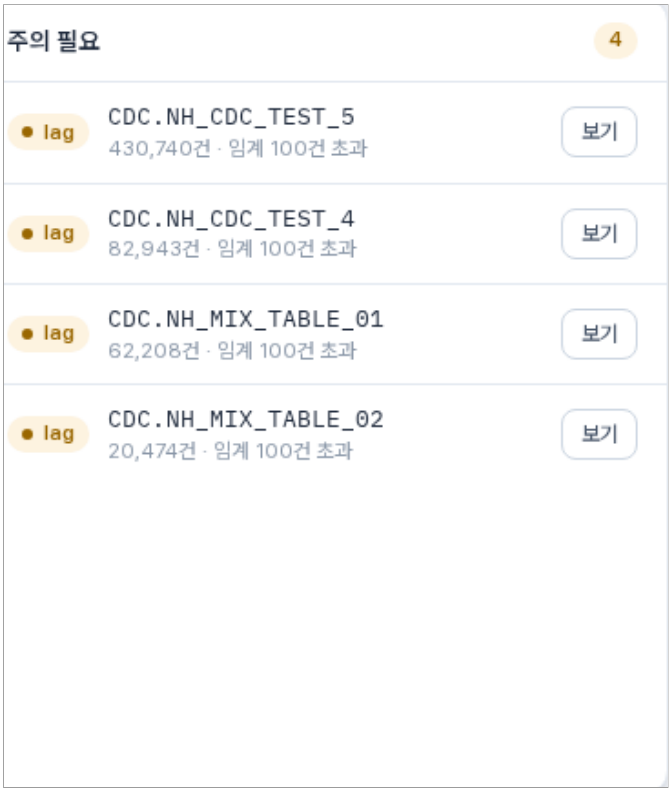


그림 E. 대시보드 "주의 필요" 목록(확대) — lag 초과 테이블을 원인 배지와 함께 즉시 노출.

3.7 단계형 재스냅샷 — 승인·홀드가 있는 상태 기계

재스냅샷을 블랙박스 버튼이 아니라 관찰 가능한 단계로 만들었다: 유입 차단 → 파이프 잔량 소진(실시간 lag) → 타깃 비우기 승인(시스템 실행은 권한 사전 점검, 직접 실행은 DBA 홀드 — 테이블별 count=0 폴링으로 자동 진행) → 재기동 → 스냅샷 → go-live. 되돌릴 수 없는 지점(offset 삭제) 전까지는 취소·실패 시 원복이 보장되고, 스냅샷 진행률은 Debezium notification 채널을 소비해 추정이 아닌 실측으로 표시한다.

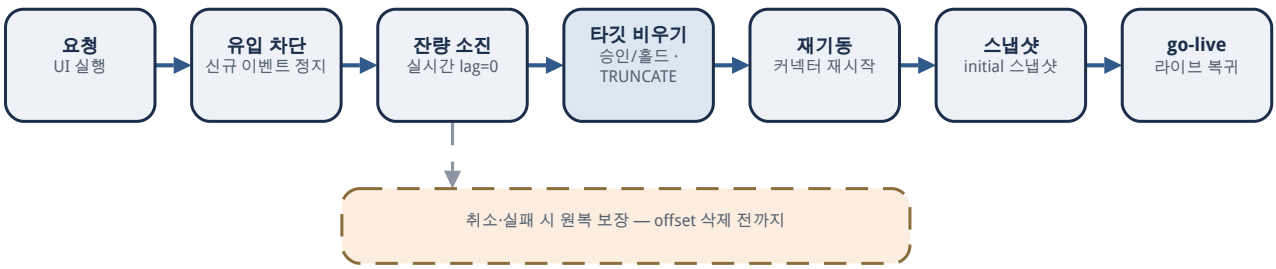


그림 F. 재스냅샷 상태 기계 — 7단계, 점선 이전(잔량 소진까지)은 취소해도 원복 보장.

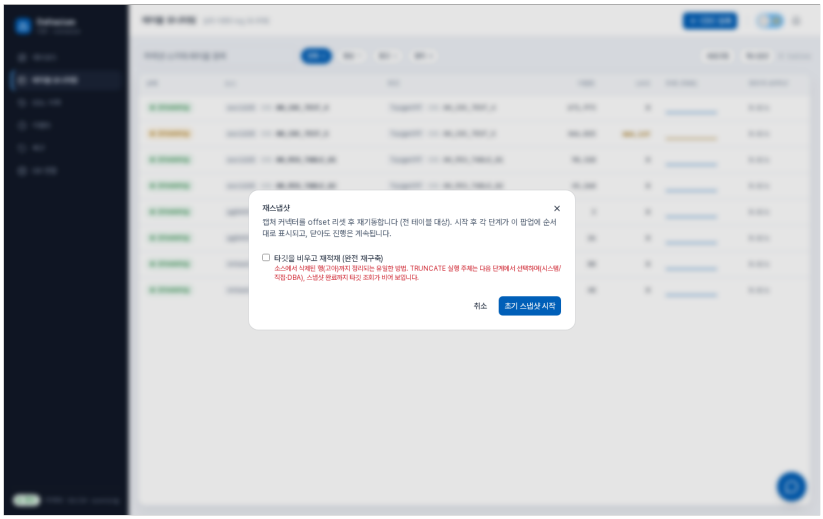


그림 G. 재스냅샷 실행 팝업(실장애 복구 run 실측) — 단계가 순서대로 표시되며 10.5만 행·50초에 go-live.

3.8 모니터링: 자체 경량 수집 → 2단 롤업

1분 샘플러(토픽 offset 델타·sink lag·pid 기반 /proc CPU·RSS)를 PostgreSQL에 적재하고, 완결 구간만 대상으로 분(48h) → 시간(60일) → 일(1년) 롤업한다. NOT EXISTS 멱등이라 재실행 중복이 없고, 집계는 지표 성격대로 처리량은 합, lag은 구간 최대로 스파이크를 보존한다. Prometheus/Grafana는 의도적으로 미도입(백로그) — 무엇을 보여줘야 하는가를 먼저 검증했다.

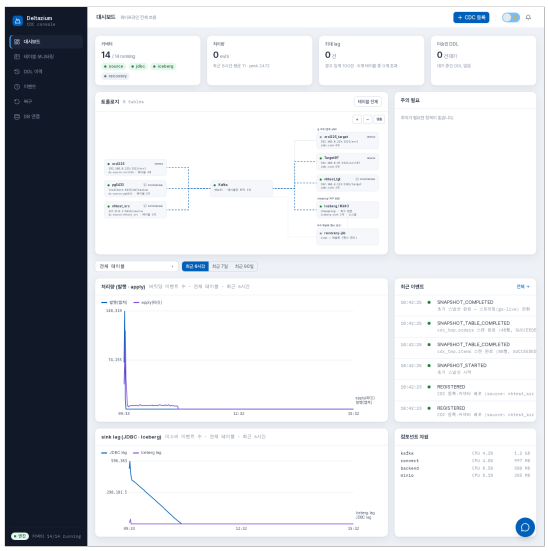


그림 H. 처리량·lag·자원 시계열(실측) — 재스냅샷 구간에 발행이 먼저, apply가 뒤따르는 형태.

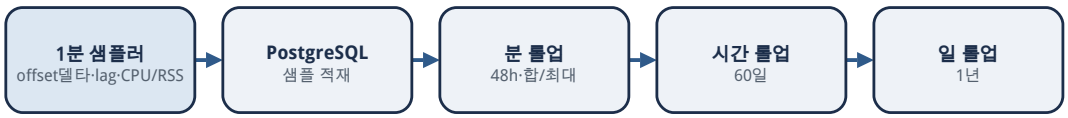


그림 I. 2단 롤업 흐름 — 완결 구간만 집계, 재실행은 NOT EXISTS로 멱등.

3.9 재개 지점(offset SCN) 유실 — 같은 예외, 세 경로의 방어

같은 종착점(`LogFileNotFoundException`)의 장애가 서로 다른 세 경로로 실발생했다: 커넥터 다운 중 archive 정리, 유휴 트래픽 중 offset 정체(Debezium offset은 이벤트 발행 시에만 영속화되므로 "유휴 기간 > 보존 기간"이면 결정론적으로 재현), 미커밋 장기 트랜잭션의 시작 SCN pin (heartbeat로도 전진 불가 — 후속 트랜잭션 처리는 막히지 않고 재시작 지점만 잡힌다). 경로별 방어를 공식 문서 검증 후 적용했다: `heartbeat.interval.ms` 60s(유휴) · archive 보존 기간(다운타임) · `log.mining.transaction.retention.ms` 2h(장기 트랜잭션 — 버려진 트랜잭션이 이후 커밋되면 복제에서 빠지는 대가를 문서화하고 선택). 롤오버 index가 시각 순서와 무관한 회전 슬롯이라는 함정 등 진단 과정의 발견도 장애 기록으로 남겼다.

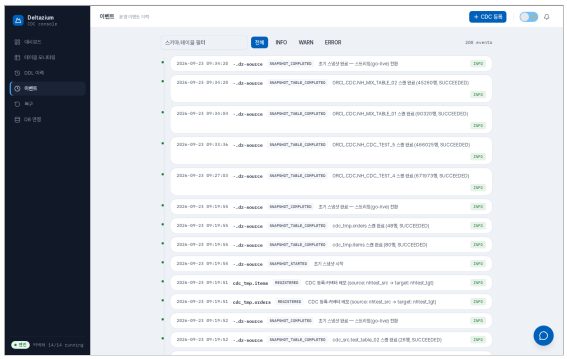


그림 J. 운영 이벤트 타임라인(실측) — 커넥터 장애 전이·복구가 테이블별로 기록된다.

3.10 AI 진단 어시스턴트 — tool use 기반 임베디드 에이전트

"왜 CDC가 멈췄어?" 질문을 받으면 Claude가 읽기 전용 도구 2종(파이프라인 상태 요약 · 로그 검색)을 스스로 골라 호출해 원인을 짚는다 — 실시간 상태 진단이라 RAG가 아닌 tool use를 선택했다. 도구 표면은 AI용으로 별도 설계(비정상만 상세 · 서버측 응답 캡 · 근거 위치 포함)하고 같은 프로세스에서 직접 실행하며, 조치 실행 도구는 의도적으로 두지 않았다 — 제안까지만, 실행은 사람이 화면에서 한다. 모든 주장에 로그 파일·줄 번호 근거를 강제해 답변을 grep으로 검증할 수 있다. tool use 왕복의 반복 전송을 prompt caching으로 흡수해 진단 1건 비용을 $1 → $0.06으로 낮췄다(실측 — 왕복별 토큰 로깅으로 캐시 적중 회귀 감시).

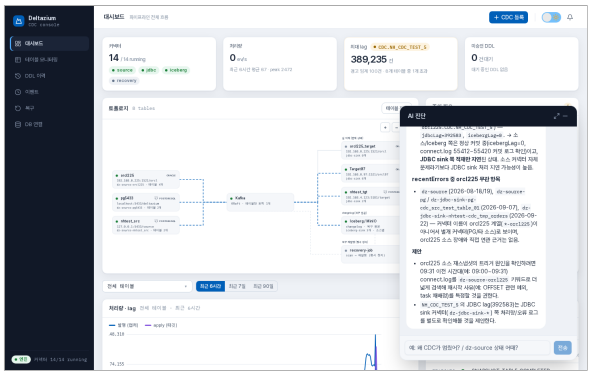


그림 K. AI 진단 위젯(실측 대화) — 로그 검색 도구를 자율 호출해 근거와 함께 원인 제시.

3.11 다중 소스·타깃 — changelog 중립 계약과 타깃 식별자 폴딩

소스 N개(Oracle·PostgreSQL)·타깃 M개를 "소스 테이블 1 → changelog 1 → 타깃 테이블 M" fan-out 하나로 한정했다(소스 fan-in은 범위 밖). 소스 식별자는 Debezium topic.prefix 하나로 충분하고, 하류(recovery-job·backend 수렴)가 `source` 블록 내부 필드(scn·lsn 등)를 읽지 않고 op/before/after/`_pos`만 본다는 changelog 중립 불변식이 소스가 늘어도 그대로 지탱한다. topic prefix는 사용자가 값을 정하지 않고 커넥션 이름 슬러그로 고정해, 이름을 바꿔도 토픽·changelog namespace·복제 슬롯 재등록이 필요 없다.

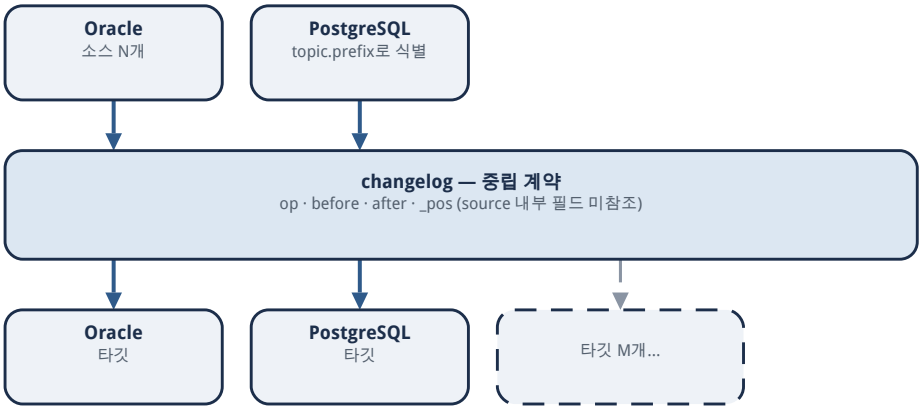


그림 L. 소스 N개·타깃 M개가 changelog 중립 계약 위에서 만난다 — 하류는 source 내부 필드를 읽지 않는다.

타깃 쪽은 PG→PG 실측(2026-09-22)에서 결함을 드러냈다 — 등록 시 타깃 스키마·테이블명을 DbType과 무관하게 대문자로 저장하던 코드가, JDBC sink는 unquoted 실행이라 PostgreSQL이 소문자로 접어 우연히 동작했지만 DDL 승인 초안은 따옴표를 쳐 존재하지 않는 대문자 스키마를 찾아 502로 실패했다. 수정은 저장 시점에 "타깃 DbType이 unquoted 식별자를 접는 형태"로 접는 함수 하나 — 소스 딕셔너리 조회용 정규화(mixed-case 소스 테이블 대응)와는 역할이 달라 기존 메서드를 고치지 않고 분리했다.

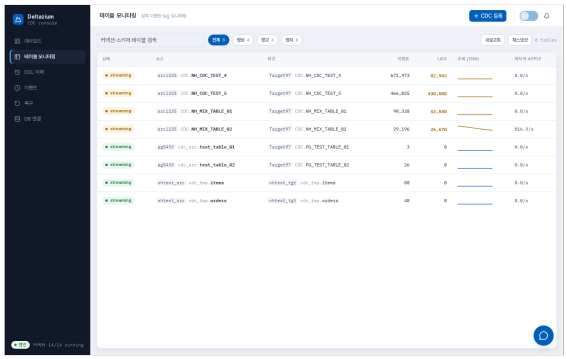


그림 M. 다중 소스·타깃 테이블 모니터링(실측) — orcl225·pg5433·nhtest_src 등 소스·타깃 연결명을 병기.

3.12 CI 구축과 CDC E2E — 시나리오가 아닌 실행 조건으로 job을 나눈다

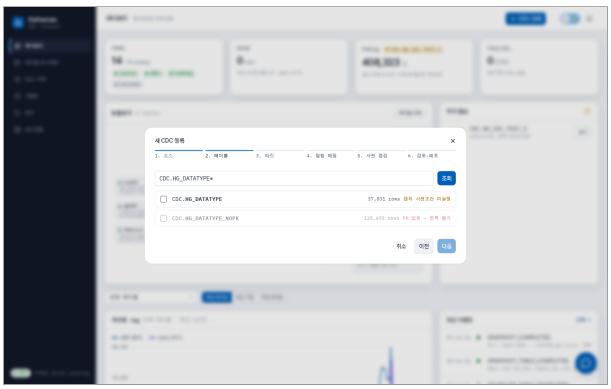
개발 서버 한 대에 GitLab CE(전용 OS 계정의 rootless podman)와 계정별 shell 러너를 올렸다(2026-10-01). 같은 PC 위의 VM은 가용성 격리도 자원 증가도 없어 별도 VM을 두지 않았고, 러너 프로세스 하나는 고정된 OS 유저로만 job을 돌리므로 각 제품의 빌드가 자기 계정 홈의 런타임·툴체인에 붙도록 계정별로 뒀다. 파이프라인은 build→test→integration: 단위 테스트(H2)는 push마다, 실 스택 E2E는 main·스케줄에서만 `resource_group`으로 직렬화해 돈다. 시나리오는 job이 아니라 스크립트 안에서 늘린다.

E2E(`ci/e2e-cdc.sh`)는 backend API만으로 연결 등록→사전 점검→등록→스냅샷 대기→소스 DML →lag 0 대기→체크섬 정합→changelog 수 검증을 거친다. at-least-once 전제로 정합은 최종 상태(체크섬) 비교, changelog는 "이벤트 수 이상" 판정. 정리는 `dze2e` prefix 객체만(가드 불일치면 FAIL, 실패 시 보존). 한계: 배포 stage 전까지는 떠 있는 backend를 테스트한다.

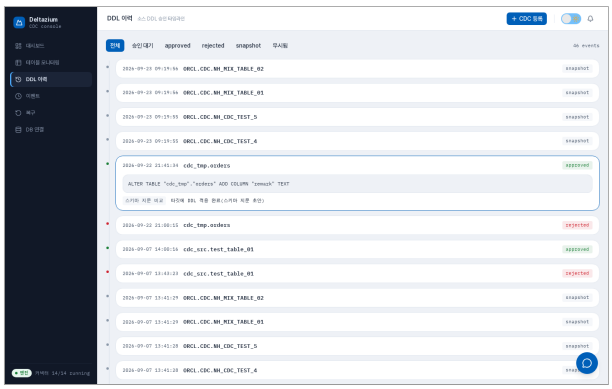
4. 웹 콘솔 기능

화면	기능
대시보드	KPI 카드(커넥터 — 클릭 시 배포된 커넥터 목록 팝오버 · 처리량 · 최대 lag · 미승인 DDL) + 동적 토폴로지(자체 SVG, 소스·타깃 연결 수만큼 노드 생성, 팬·줌·노드 클릭 정보 카드 — 교차 없는 직교 라우팅·흐름 애니메이션) + 주의 필요 목록(lag 초과·DDL 대기, 해당 화면 이동) + 처리량(발행 vs apply)·lag 시계열(테이블·기간 선택) + 최근 운영 이벤트
CDC 등록 위저드	6단계: 소스 선택 → 딕셔너리 조회(와일드카드 전개, PK 없는 테이블 선택 차단) → 타깃 → 컬럼 매핑(제외·변환식·구문 검증) → 사전 점검(supp.log/REPLICA IDENTITY 승인 후 적용 · 권한 GRANT 안내) → 스냅샷 모드 선택·배포
테이블 모니터링	상태 필터 칩 + 그리드(소스·타깃 연결명 병기 — 다중 소스·타깃 구분, 실측 이벤트·lag·15분 추세), 행 선택 시 상세 drawer(상태·lag 차트·이벤트 이력·정지/재개/재시도/재스냅샷/삭제), 캡처 장애 배너(원인·trace·복구 진입)
DDL 이력	schema change topic 상시 수집(retention 만료 대비 DB 적재) + PostgreSQL 소스는 스키마 지문 비교로 변경 감지(schema change topic이 없어 방식이 다름), 승인(타깃 이름 치환 실행)/거부(테이블 단위 apply 정지), 비전파성 DDL 자동 무시
이벤트	테이블별 운영 이력 — 등록·정지·DDL·복구·커넥터 장애 전이(원인 trace 포함) 타임라인
복구	changelog(S3) 현황(용량·보존 구간·파티션별)·실행 이력 + 단계형 실행 drawer(대상 테이블 → 재발행 범위 → 정합 검증) · go-live 자동 재개 · SRC/TGT 정합 검증(행수+체크섬)
DB 연결	소스/타깃 연결 카드(DB 종류 로고 표시) + changelog 저장소 카드 — 접속 정보·사용 중인 테이블 수, 인라인 연결 테스트(카드 액센트로 결과 표시)
헤더 알림	전 화면 공통 — 경고 배지(커넥터 장애 등 CRITICAL/WARN)와 알림 벨(사용자가 직접 정지했거나 DDL 거부로 apply만 멈춘 것 — INFO, 확인 후 제거)을 분리해 "고장"과 "의도된 정지"를 구분 표시
AI 진단 위젯	전 화면 공통 플로팅 채팅 — 상태 요약·로그 검색 도구 자율 호출, 로그 라인 근거와 함께 원인 제시(SSE 스트리밍·사용 도구 표시), 조치는 제안만

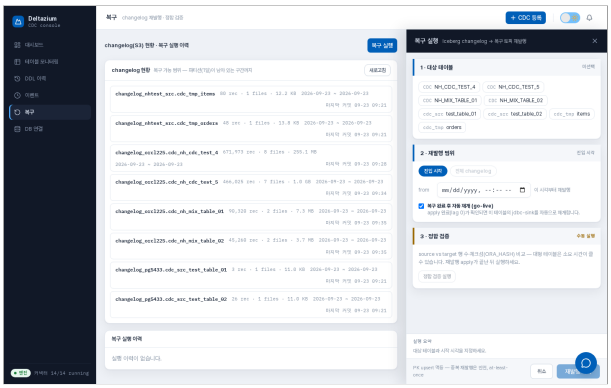
아래는 위 기능 중 본문에 싣지 않은 화면이다.



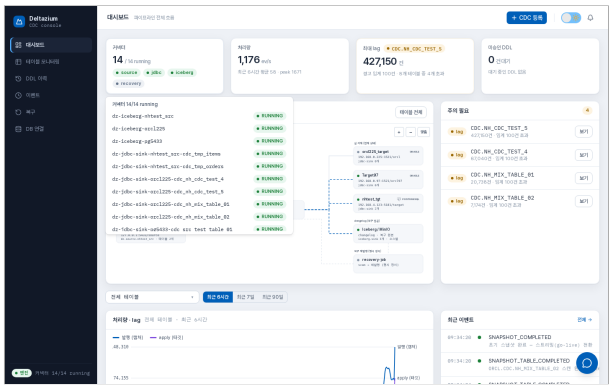
등록 위저드 — PK 없는 테이블 선택 차단, 캡처 사전조건 미설정 경고



DDL 이력 — PG 소스 스키마 지문(FINGERPRINT) 감지로 ALTER 승인 사례



복구 — changelog 현황·복구 실행 drawer·go-live 자동 재개



커넥터 KPI 카드 클릭 — 배포된 커넥터 14개 상태 목록 팝오버

5. 검증 결과

항목	내용
실 Oracle end-to-end	초기 스냅샷(30,088행) → LogMiner 실시간 스트리밍 → 타깃 apply(lag 0) + changelog 커밋까지 실트래픽으로 확인
캡처 장애 전환	ORA-1371 재현 → online_catalog 전환 → 밀린 9,673건 유실 0 회수
자동화 테스트 136건	envelope 왕복 동등성 · changelog 사전 생성(실 카탈로그) · 복구 재발행 스모크(73건) · 로그 검색·상태 요약·chat 단위 테스트 포함
사전 점검 게이트	LogMiner 권한 누락 감지 → GRANT 안내 → 부여 후 통과 (실동작)
실장애 복구	archive log 소실로 죽은 캡처(나흘 경과)를 재스냅샷 워크플로로 복구 — truncate 완전 재구축 10.5만 행·50초 go-live, 진행률 notification 실측
정합 검증	행 수 + ORA_HASH 체크섬으로 고아 행 편차(+2,237) 검출 → 재구축 후 수렴 확인
SCN 유실 방어 실증	heartbeat 적용 후 heartbeat 토픽 1,459건/일 발행 확인, 세 번째 경로(장기 트랜잭션 pin) 재발을 로그로 원인 확정 후 retention 적용
AI 진단 실전	재발 장애(offset SCN 유실)를 위젯 질문 한 번으로 원인 확정 — connect.log 라인 근거 인용, 실패 1시간 전 선행 경고(장기 트랜잭션)까지 로그에서 발굴

모니터링 실측	분당 2,262건 버스트에서 발행·apply 추적, lag 스파이크·소진 시계열 기록, 롤업 멱등 테스트
다중 소스·타깃 실측	PostgreSQL 소스 추가(②, 2026-09-07) 검증 통과. PG→PG 시나리오(2026-09-22, docs/experiments/2026-09-22-pg2pg-cdc.md): 스냅샷 30/60건 → DML insert 50·update 25·delete 15건 → 타깃 45/80건까지 정렬 덤프 diff 0, DDL 스키마 지문 감지 정상. 타깃 식별자 대문자 정규화로 PG 타깃 DDL 승인이 502로 실패하는 결함(D1) 발견 → 타깃 DbType 기준 폴딩으로 수정 → 재검증 성공 (orders 47건, diff 0). topic prefix는 커넥션 이름 슬러그로 고정(4절).
정합 검증 체크섬 정밀화 (2026-09-23)	컬럼별 해시로 바꾼 체크섬이 CDC.NH_CDC_TEST_5(206컬럼, 466,025행) 중 정확히 1행 불일치를 검출 — 1951-05-06 KST DST 공백 시각의 DATE가 Connect 워커 JVM의 로컬 타임존 해석으로 1시간 밀려 적재된 것이 원인(docs/incidents/2026-09-23-dst-gap-date-shift.md). Connect 워커 JVM 타임존을 UTC로 고정해 수정, 재검증에서 466,025행 체크섬 전량 일치. 같은 작업에서 PostgreSQL 체크섬 지원과 등록 해제 시 PG 복제 슬롯·publication 자동 정리도 반영했다.
CDC E2E 자동화 (2026-10-02)	PG→PG 전 구간 스크립트가 실 스택에서 PASS(114초) — 스냅샷 3행 + DML 6건 → 타깃 체크섬 일치, changelog 9건 이상, 정리 후 잔여 객체 0. 실패 보존·`--cleanup-only` 확인. backend 단위 테스트 247건 통과. CI integration stage에 연결(3.12절).

6. 기술 스택 · 문서화

Java 21 · Spring Boot 3.5 · MyBatis · Debezium 3.6 (Oracle LogMiner · PostgreSQL pgoutput · JDBC sink) · Kafka 4.3 KRaft(ZooKeeper 없는 단일 노드) · Apache Iceberg 1.11 · MinIO(S3) · PostgreSQL 15 · React 19 · TypeScript · Vite · Tailwind 4 · shadcn/ui · TanStack Table · 자체 SVG(토폴로지·차트) · Claude API(tool use · SSE).

설계 기준(architecture.md) · 운영 가이드(operations.md) · 구현 내부 노트(internals.md) · CI 구축 기록(ci.md) · 실장애 기록(incidents/, 건별 타임라인·진단 경로·교훈) · 실측 실험 기록(experiments/)을 리포에 유지 — "UI에는 사용자 결정에 필요한 것만, 시스템 내부는 문서·주석에" 원칙. 문서는 작업이 아니라 주제 단위로 갱신한다: 새 작업이 생겨도 새 md를 만들지 않고 기존 문서의 해당 절을 고친다.

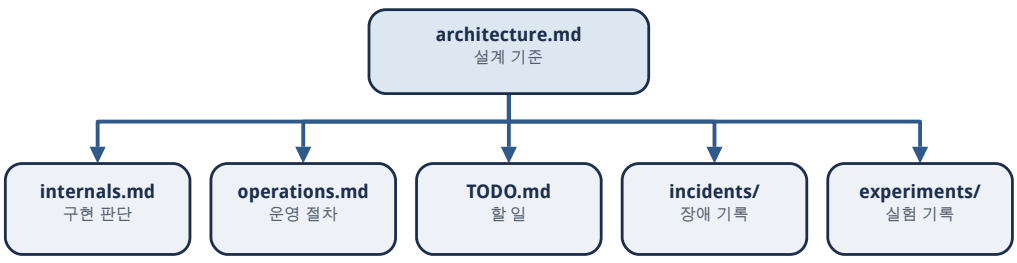


그림 0. 문서 체계 — 설계 기준 아래로 구현·운영·할 일·장애·실험 기록이 갈라진다.

남은 과제: 기동 중 테이블 추가 시 초기적재(incremental snapshot + Kafka signal) · Prometheus/Grafana(JMX exporter) 도입 후 자체 수집과 비교 · 컬럼 리네임의 적재 반영(스톡 sink 한계 — 정책 결정 대기) · S3 복구 풀 리허설 실행 검증 · AI 진단 멀티턴(후속 질문 맥락 유지) · offset 정체 조기 경보(실패 전 WARN의 운영 이벤트 승격) · CI 배포 stage · Oracle 소스 E2E · changelog 요약 API

부록 A. 콘솔 화면

개발 환경 화면 캡처이며, 내부 주소 등 일부 정보는 가림 처리했다. 본문에 실은 화면은 제외했다.

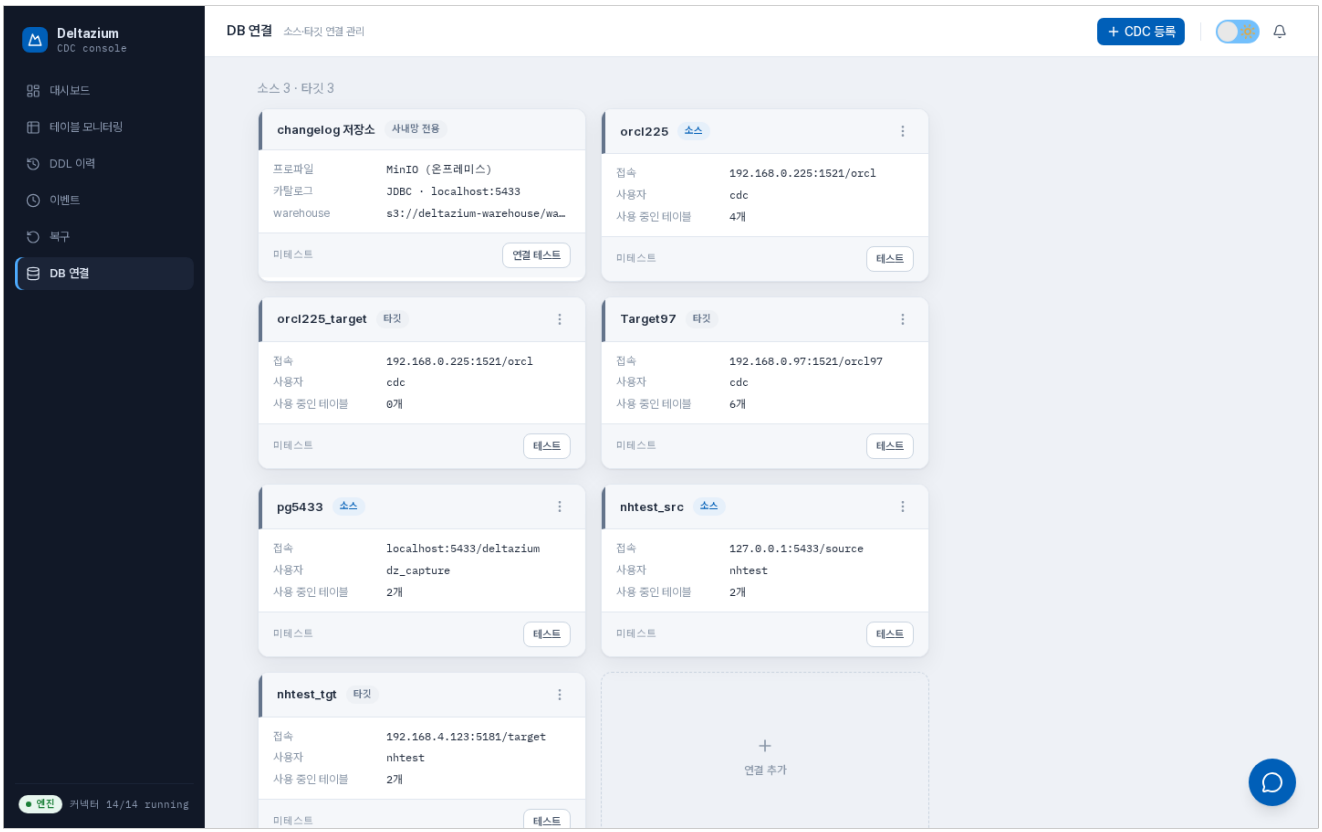


그림 N. DB 연결 — 소스 3·타깃 3 + changelog 저장소(MinIO) 카드, 접속 정보와 사용 중인 테이블 수를 한눈에 확인